

PyGAGE User Manual

Generally Applicable Gene Set Enrichment Analysis in Python

[Version 1.0]

[Date: February 2026]

Introduction

Installation

Quick Start

Core Modules

Detailed Function Reference

Data Formats

Complete Workflows

Visualization

Advanced Topics

Troubleshooting

References

1. Introduction

What is PyGAGE?

PyGAGE (Python implementation of Generally Applicable Gene Set Enrichment) is a comprehensive toolkit for gene set enrichment analysis. It implements the GAGE method, which tests whether sets of genes are significantly enriched in an expression dataset compared to the background.

Key Features

- [General Applicability]: Works with various experimental designs and sample sizes
- [Robust Statistics]: Multiple test methods (t-test, z-test, Kolmogorov-Smirnov)
- [Flexible Comparisons]: Paired, unpaired, one-on-group, and as-group comparisons
- [High Performance]: Built on Polars for fast data processing
- [Modern Visualization]: Seaborn/matplotlib for publication-quality plots
- [Dual Interface]: Both command-line and Python API

What Makes GAGE Different?

Unlike traditional gene set enrichment methods:

- [No arbitrary thresholds]: Tests gene sets directly without pre-filtering genes
- [Sample-size independent]: Works with 1 to 1000+ samples
- [Directional testing]: Can test for up-regulation, down-regulation, or both
- [Flexible experimental designs]: Handles paired, unpaired, and heterogeneous data

Citation

If you use PyGAGE, please cite:

] Luo, W., Friedman, M., Shedden K., Hankenson, K. and Woolf, P. (2009)
] GAGE: Generally Applicable Gene Set Enrichment for Pathway Analysis.
] BMC Bioinformatics 10:161

2. Installation

Requirements

- Python 3.8 or higher
- pip package manager

Install Dependencies

```
pip install polars numpy scipy matplotlib seaborn requests
```

Install PyGAGE

Download the PyGAGE package and add to your Python path:

```
# Option 1: Add to PYTHONPATH
export PYTHONPATH="/path/to/pygage:$PYTHONPATH"

# Option 2: Install in development mode
cd /path/to/pygage
pip install -e .
```

Verify Installation

```
import polars as pl
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns

print("All dependencies installed successfully!")
```

3. Quick Start

Basic Workflow

```
from gage_core import GAGEPreparation, GAGEAnalysis
import polars as pl
import json

# 1. Load expression data (genes × samples)
expr_data = pl.read_csv('expression_data.csv')

# 2. Load gene sets
with open('kegg_pathways.json') as f:
    gene_sets = json.load(f)

# 3. Define sample groups
ref_indices = [0, 1, 2] # Control samples (columns 0-2)
samp_indices = [3, 4, 5] # Treatment samples (columns 3-5)

# 4. Prepare data
prep = GAGEPreparation()
prepared = prep.prepare_expression(
    expr_data,
    ref_indices=ref_indices,
    samp_indices=samp_indices,
    comparison='paired'
)

# 5. Run GAGE analysis
gage = GAGEAnalysis()
results = gage.run_gage(
    prepared,
    gene_sets,
    gene_col='gene_id',
    test_method='t-test'
)

# 6. Filter significant gene sets
significant = gage.filter_significant(cutoff=0.1)

# 7. View results
print(significant['greater'].head()) # Up-regulated pathways
print(significant['less'].head())   # Down-regulated pathways
```

Command-Line Quick Start

```
# Run complete analysis
```

```
python gage_core.py \  
    --expression data.csv \  
    --gene-sets pathways.json \  
    --ref-indices 0 1 2 \  
    --samp-indices 3 4 5 \  
    --comparison paired \  
    --cutoff 0.1 \  
    --output results/
```

```
# Results written to:  
# - results/greater.tsv (up-regulated gene sets)  
# - results/less.tsv (down-regulated gene sets)  
# - results/stats.tsv (test statistics)
```

4. Core Modules

Module Overview

5. Detailed Function Reference

5.1 Gene ID Utilities

GeneIDConverter Class

[Purpose]: Convert between Entrez Gene IDs and gene symbols.

[Initialization:]

```
from gene_id_utils import GeneIDConverter

converter = GeneIDConverter('egSymb.csv')
# Mapping file format: entrez_id,symbol
```

[Methods:]

```
##### "eg2sym(entrezids)"
```

Convert Entrez Gene IDs to official gene symbols.

[Parameters:]

- "entrezids" (list): List of Entrez Gene IDs (strings or integers)

[Returns:]

- List of gene symbols (None for missing IDs)

[Example:]

```
symbols = converter.eg2sym(['1', '2', '3'])
# Returns: ['TP53', 'BRCA1', 'EGFR'] (or similar)
```

```
##### "sym2eg(symbols)"
```

Convert gene symbols to Entrez Gene IDs.

[Parameters:]

- "symbols" (list): List of gene symbols

[Returns:]

- List of Entrez Gene IDs (None for missing symbols)

[Example:]

```
entrez_ids = converter.sym2eg(['TP53', 'BRCA1', 'EGFR'])  
# Returns: ['7157', '672', '1956'] (or similar)
```

[Command-Line Usage:]

```
python gene_id_utils.py input.txt \  
    --mapping egSymb.csv \  
    --direction eg2sym \  
    --output output.csv
```

5.2 Pathway Database Utilities

KEGGPathwayRetriever Class

[Purpose]: Retrieve up-to-date KEGG pathway gene sets.

[Methods:]

```
##### "getpathwaygenes(species='hsa', idtype='kegg')"
```

Generate KEGG pathway gene sets for any KEGG species.

[Parameters:]

- "species" (str): KEGG species code. Options:
 - "hsa" - Human (Homo sapiens)
 - "mmu" - Mouse (Mus musculus)
 - "rno" - Rat (Rattus norvegicus)
 - "dme" - Fruit fly (Drosophila melanogaster)
 - "cel" - C. elegans
 - "eco" - E. coli
 - "ko" - KEGG Ortholog
 - See KEGG for full list
- "idtype" (str): Gene ID type
 - "kegg" - KEGG gene IDs (default)
 - "entrez" - Entrez Gene IDs

[Returns:]

Dictionary with:

- "genesets": Dict mapping pathway IDs to gene lists
- "pathwaynames": Dict mapping pathway IDs to names
- "categories": Dict with 'signal', 'metabolic', 'disease' pathway indices

[Example:]

```
from pathway_database_utils import KEGGPathwayRetriever

kegg = KEGGPathwayRetriever()
pathways = kegg.get_pathway_genes(species='hsa', id_type='entrez')

# Access gene sets
print(pathways['gene_sets']['00010']) # Glycolysis pathway genes
print(pathways['pathway_names']['00010']) # Pathway name

# Filter to signaling pathways only
signal_pathways = {
    k: pathways['gene_sets'][k]
    for k in pathways['categories']['signal']
}
```

[Command-Line Usage:]

```
python pathway_database_utils.py kegg \
    --species hsa \
    --id-type entrez \
    --output kegg_pathways.json
```

GOGeneSetRetriever Class

[Purpose]: Retrieve Gene Ontology gene sets.

```
##### "getgogenesets(species='human', annotationfile=None)"
```

Retrieve GO gene sets from a GAF annotation file.

[Parameters:]

- "species" (str): Species name
- "annotationfile" (Path): GO annotation file in GAF format

[Returns:]

Dictionary with:

- "genesets": Dict mapping GO terms to gene lists
- "gocategories": Dict categorizing GO terms (BP, MF, CC)

- "gonames": Dict mapping GO terms to descriptions

[Example:]

```
from pathway_database_utils import GOGeneSetRetriever

go = GOGeneSetRetriever()
gene_sets = go.get_go_gene_sets(
    species='human',
    annotation_file='goa_human.gaf'
)

# Access GO gene sets
bp_sets = gene_sets['go_categories']['BP'] # Biological Process
mf_sets = gene_sets['go_categories']['MF'] # Molecular Function
cc_sets = gene_sets['go_categories']['CC'] # Cellular Component
```

[Note:] Download GO annotation files from:

- <http://geneontology.org/docs/download-go-annotations/>

5.3 GAGE Core Analysis

GAGEPreparation Class

[Purpose]: Prepare expression data for GAGE analysis.

```
##### "prepareexpression(data, refindices, sampindices, comparison, samedir, usefold)"
```

Prepare expression data by calculating per-gene statistics.

[Parameters:]

- "data" (pl.DataFrame): Expression matrix (genes × samples)
- "refindices" (list[int]): Reference/control sample column indices
- "sampindices" (list[int]): Treatment/experimental sample column indices
- "comparison" (str): Comparison method
- ""paired"" (default): One-to-one paired samples
- ""unpaired"": All possible pairs between ref and samp
- ""1ongroup"": Each samp vs mean of all ref
- ""as.group"": Mean of samp vs mean of ref
- "samedir" (bool): Test for same direction (default: True)

- "True": Test up and down separately
- "False": Test both directions together
- "usefold" (bool): Use fold change (log ratio) vs raw difference (default: True)

[Returns:]

- Prepared DataFrame ready for gene set testing

[Example:]

```
from gage_core import GAGEPreparation
import polars as pl

# Load data
expr_data = pl.read_csv('expression_data.csv')

# Prepare for paired comparison
prep = GAGEPreparation()
prepared = prep.prepare_expression(
    expr_data,
    ref_indices=[0, 1, 2],      # Control: columns 0-2
    samp_indices=[3, 4, 5],     # Treatment: columns 3-5
    comparison='paired',       # Paired design
    same_dir=True,              # Test up/down separately
    use_fold=True               # Use log fold changes
)
```

GAGEAnalysis Class

[Purpose]: Run GAGE enrichment analysis.

```
##### "rungage(expressiondata, genesets, genecol, setsizerange, samedir, testmethod, fdmethod)"
```

Run GAGE analysis on prepared expression data.

[Parameters:]

- "expressiondata" (pl.DataFrame): Prepared expression data from "prepareexpression()"
- "genesets" (dict): Gene sets as {setname: [genelist]}
- "genecol" (str): Gene ID column name (default: 'geneid')
- "setsizerange" (tuple): Min and max gene set size (default: (10, 500))
- Sets outside this range are excluded from analysis
- Prevents testing very small (unstable) or very large (uninformative) sets
- "samedir" (bool): Test for same direction (default: True)

- "testmethod" (str): Statistical test to use
- "t-test" (default): Two-sample t-test - most common
- "z-test": Z-test (similar to PAGE method)
- "ks-test": Kolmogorov-Smirnov test (similar to GSEA)
- "fdrmethod" (str): FDR correction method (default: 'BH')
- "BH": Benjamini-Hochberg procedure

[Returns:]

Dictionary with:

- "greater": DataFrame of up-regulated gene sets
- "less": DataFrame of down-regulated gene sets (if samedir=True)
- "stats": DataFrame of test statistics

[Output Columns:]

[Example:]

```
from gage_core import GAGEAnalysis
import json

# Load gene sets
with open('kegg_pathways.json') as f:
    gene_sets = json.load(f)['gene_sets']

# Run GAGE
gage = GAGEAnalysis()
results = gage.run_gage(
    prepared_data,
    gene_sets,
    gene_col='gene_id',
    set_size_range=(10, 500),
    same_dir=True,
    test_method='t-test'
)

# View top up-regulated pathways
print(results['greater'].sort('q_greater').head(10))

# View top down-regulated pathways
print(results['less'].sort('q_less').head(10))
```

"filtersignificant(cutoff, useq)"

Filter results to significant gene sets only.

[Parameters:]

- "cutoff" (float): Significance cutoff (default: 0.1)
- For q-values: typically 0.05, 0.1, or 0.25
- For p-values: typically 0.001 or 0.01
- "useq" (bool): Use q-value (True) or p-value (False) (default: True)

[Returns:]

Filtered dictionary with same structure as "rungage()" results

[Example:]

```
# Filter at q < 0.05
significant = gage.filter_significant(cutoff=0.05, use_q=True)

print(f"Significant up: {significant['greater'].shape[0]}")
print(f"Significant down: {significant['less'].shape[0]}")
```

5.4 Statistical Tests

GeneSetTests Class

[Purpose]: Perform statistical tests on gene sets.

```
##### "ttest(expressiondata, genesets, genecol, setsizerange, samedir)"
```

Two-sample t-test for gene set enrichment.

[When to use:]

- Most common choice
- Good for normally distributed data
- Robust to moderate violations of normality

[Parameters:]

- Same as "rungage()" parameters

[Returns:]

Dictionary with test results

[Example:]

```
from gage_tests import GeneSetTests
```

```

tester = GeneSetTests()
results = tester.t_test(
    expr_data,
    gene_sets,
    set_size_range=(10, 500)
)

# Results DataFrame with p-values and statistics
results_df = results['results']

```

"ztest(expressiondata, genesets, ...)"

Z-test for gene set enrichment (similar to PAGE method).

[When to use:]

- Large sample sizes ($n \geq 30$)
- Known population variance
- Faster than t-test for large datasets

"kolmogorovsmirnovtest(expressiondata, genesets, ...)"

Non-parametric Kolmogorov-Smirnov test (similar to GSEA).

[When to use:]

- Non-normal data distributions
- Ranked data
- Distribution-free testing

[Note:] Automatically ranks data before testing.

5.5 Data Processing Utilities

DataTransformer Class

"rownormalize(data)"

Normalize rows to z-scores (mean=0, sd=1).

[Parameters:]

- "data" (pl.DataFrame): Expression data

[Returns:]

- Row-normalized DataFrame

[Use case:] Prepare data for heatmap visualization

[Example:]

```
from data_processing_utils import DataTransformer

transformer = DataTransformer()
normalized = transformer.row_normalize(expr_data)
```

GeneExtractor Class

```
##### "extractessentialgenes(geneset, expressiondata, genecol, threshold, rankbyabs)"
```

Extract essential (highly differential) genes from a gene set.

[Parameters:]

- "geneset" (list): List of gene IDs to consider
- "expressiondata" (pl.DataFrame): Expression data
- "genecol" (str): Gene ID column name
- "threshold" (float): Z-score threshold (default: 1.0)
- 1.0 = genes]1 SD from mean
- 2.0 = genes]2 SD from mean (more stringent)
- "rankbyabs" (bool): Rank by absolute value (default: False)

[Returns:]

- DataFrame with essential genes only

[Use case:] Identify key genes driving pathway enrichment

[Example:]

```
from data_processing_utils import GeneExtractor

extractor = GeneExtractor()

# Extract essential genes from top pathway
top_pathway = significant['greater'][0, 'gene_set']
pathway_genes = gene_sets[top_pathway]

essential = extractor.extract_essential_genes(
    pathway_genes,
```



```

        expr_data,
        gene_col='gene_id',
        threshold=2.0 # Only highly differential genes
    )

    print(f"Essential genes: {essential.shape[0]}")

```

GeneDataExporter Class

```
##### "exportgenedata(genes, expressiondata, outputfile, createheatmap, heatmapoutput, normalize)"
```

Export and visualize gene expression data.

[Parameters:]

- "genes" (list): Gene IDs to export
- "expressiondata" (pl.DataFrame): Expression data
- "outputfile" (Path): Output CSV/TSV file
- "createheatmap" (bool): Generate heatmap (default: False)
- "heatmapoutput" (Path): Heatmap image file
- "normalize" (bool): Row-normalize for heatmap (default: True)

[Example:]

```

from data_processing_utils import GeneDataExporter
from pathlib import Path

exporter = GeneDataExporter()
exporter.export_gene_data(
    genes=essential['gene_id'].to_list(),
    expression_data=expr_data,
    output_file=Path('essential_genes.csv'),
    create_heatmap=True,
    heatmap_output=Path('essential_heatmap.png'),
    normalize=True
)

```

5.6 Results Analysis

ResultsComparator Class

```
##### "compareresults(resultfiles, samplenames, qcutoff, outputfile)"
```

Compare GAGE results across multiple experiments.

[Parameters:]

- "resultfiles" (list[Path]): Result files to compare
- "samplenames" (list[str]): Names for each experiment
- "qcutoff" (float): Q-value cutoff (default: 0.1)
- "outputfile" (Path): Output file for combined results

[Returns:]

- Combined DataFrame with all results

[Use case:] Meta-analysis across multiple studies

[Example:]

```
from results_analysis import ResultsComparator
from pathlib import Path

comparator = ResultsComparator()
combined = comparator.compare_results(
    result_files=[
        Path('study1_greater.tsv'),
        Path('study2_greater.tsv'),
        Path('study3_greater.tsv')
    ],
    sample_names=['Study1', 'Study2', 'Study3'],
    q_cutoff=0.1,
    output_file=Path('combined_results.tsv')
)

# Find pathways significant in all studies
all_sig = combined.filter(pl.col('hits') == 3)
print(f"Significant in all: {all_sig.shape[0]}")
```

"createvenncomparison(resultfiles, samplenames, qcutoff, outputfile)"

Create Venn diagram comparing significant gene sets (2-3 experiments only).

[Example:]

```
comparator.create_venn_comparison(
    result_files=[Path('exp1.tsv'), Path('exp2.tsv')],
    sample_names=['Control', 'Treatment'],
    q_cutoff=0.05,
    output_file=Path('venn_comparison.png')
)
```

SignificanceFilter Class

```
##### "filtersignificant(results, cutoff, useq, dualsig)"
```

Filter significant gene sets with handling for dual-significant sets.

[Parameters:]

- "results" (dict): GAGE results dictionary
- "cutoff" (float): Significance cutoff
- "useq" (bool): Use q-value (default: True)
- "dualsig" (int): Dual significance mode
- "0": Exclude dual-significant sets
- "1": Keep in more significant direction only
- "2": Keep in both directions (default)

[Dual-Significant Sets:]

Gene sets that are significantly perturbed in both directions (some genes up, some down). This can indicate:

- Pathway regulation complexity
- Sample heterogeneity
- Data quality issues

[Example:]

```
from results_analysis import SignificanceFilter

filterer = SignificanceFilter()
filtered = filterer.filter_significant(
    results={'greater': greater_df, 'less': less_df},
    cutoff=0.1,
    use_q=True,
    dual_sig=2 # Keep dual-significant in both directions
)
```

GeneSetGrouper Class

```
##### "groupgenesets(results, genesets, expressiondata, pcutoff, overlapcutoff, outputfile)"
```

Group overlapping/redundant gene sets based on shared genes.

[Parameters:]

- "results" (pl.DataFrame): GAGE results

- "genesets" (dict): Gene sets dictionary
- "expressiondata" (pl.DataFrame): Expression data
- "pcutoff" (float): Significance cutoff (default: 0.01)
- "overlapcutoff" (float): Overlap significance cutoff (default: 1e-10)
- "outputfile" (Path): Output JSON file

[Returns:]

- Dictionary of gene set groups

[Use case:] Reduce redundancy in significant pathways

[Example:]

```
from results_analysis import GeneSetGrouper

grouper = GeneSetGrouper()
groups = grouper.group_gene_sets(
    results=significant['greater'],
    gene_sets=gene_sets,
    expression_data=expr_data,
    output_file=Path('gene_set_groups.json')
)

# View groups
for group_name, members in groups.items():
    print(f"{group_name}: {len(members)} gene sets")
    print(f"  Members: {' '.join(members[:3])}..." )
```

5.7 Visualization Utilities

HeatmapPlotter Class

```
##### "plotheatmap(data, rowlabels, collabels, cmap, vmin, vmax, center, figsize, outputfile, title)"
```

Create a basic heatmap.

[Parameters:]

- "data" (pl.DataFrame or np.array): Data matrix
- "rowlabels" (list): Row labels
- "collabels" (list): Column labels
- "cmap" (str): Colormap name (default: 'RdYlGnr')
- "'RdYlGnr'": Red-Yellow-Green (reversed)

- "viridis": Perceptually uniform
- "coolwarm": Blue-Red diverging
- See matplotlib colormaps
- "vmin", "vmax" (float): Value range
- "center" (float): Center value for diverging colormaps
- "figsize" (tuple): Figure size in inches
- "outputfile" (Path): Output image file
- "title" (str): Plot title

[Example:]

```
from visualization_utils import HeatmapPlotter

plotter = HeatmapPlotter()
plotter.plot_heatmap(
    data=normalized_data,
    cmap='RdYlGn_r',
    center=0,
    figsize=(10, 8),
    output_file=Path('heatmap.png'),
    title='Gene Expression Heatmap'
)
```

"plotclusteredheatmap(data, ...)"

Create heatmap with hierarchical clustering dendrograms.

[Additional features:]

- Automatic row and column clustering
- Dendrograms showing relationships
- Better for discovering patterns

[Example:]

```
plotter.plot_clustered_heatmap(
    data=significant_pathways,
    cmap='RdYlGn_r',
    figsize=(12, 10),
    output_file=Path('clustered_heatmap.png'),
    title='Significant Pathways'
)
```

VennDiagram Class

```
##### "venncounts(data, include)"
```

Calculate counts for Venn diagram regions.

[Parameters:]

- "data" (pl.DataFrame): Binary data (0/1 or True/False)
- "include" (str): What to include
- "'both'": Any value]0 or [0
- "'up'": Only positive values
- "'down'": Only negative values

[Returns:]

- DataFrame with counts for each region

```
##### "plotvenn2(counts, names, outputfile, figsize)"
```

Plot 2-set Venn diagram.

[Example:]

```
from visualization_utils import VennDiagram
import polars as pl

# Create binary significance matrix
sig_matrix = pl.DataFrame({
    'Study1': [1, 1, 0, 1, 0],
    'Study2': [1, 0, 1, 1, 0]
})

venn = VennDiagram()
counts = venn.venn_counts(sig_matrix, include='both')
venn.plot_venn2(
    counts,
    names=['Study 1', 'Study 2'],
    output_file=Path('venn.png')
)
```

```
##### "plotvenn3(counts, names, outputfile, figsize)"
```

Plot 3-set Venn diagram.

[Note:] Maximum 3 sets supported for Venn diagrams.

ColorUtils Class

```
##### "createcolormap(low, mid, high, n)"
```

Create custom color gradient.

[Example:]

```
from visualization_utils import ColorUtils

# Green-Black-Red gradient
cmap = ColorUtils.create_colormap('green', 'black', 'red', n=256)

# Blue-White-Red gradient
cmap = ColorUtils.create_colormap('#0000FF', 'FFFFFF', 'FF0000', n=256)
```

```
##### "greenred(n)"
```

Create standard green-black-red colormap (for expression data).

6. Data Formats

6.1 Expression Data

[CSV/TSV Format:]

```
gene_id,sample1,sample2,sample3,sample4,sample5,sample6
GENE001,5.234,5.412,5.156,8.312,8.523,8.145
GENE002,3.145,3.321,3.234,3.412,3.534,3.321
GENE003,7.812,7.923,8.012,4.234,4.123,4.345
```

[Requirements:]

- First column: Gene IDs (must match gene sets)
- Remaining columns: Expression values (numeric)
- Gene IDs can be: Entrez IDs, gene symbols, Ensembl IDs, etc.
- Values can be: raw counts, FPKM, TPM, log2-transformed, etc.

[Loading:]

```
import polars as pl

# CSV
data = pl.read_csv('expression_data.csv')

# TSV
data = pl.read_csv('expression_data.tsv', separator='\t')
```

6.2 Gene Sets

[JSON Format:]

Simple format:

```
{
  "pathway1": ["GENE001", "GENE002", "GENE005"],
  "pathway2": ["GENE003", "GENE004", "GENE006"],
  "pathway3": ["GENE001", "GENE003", "GENE007"]
}
```

With metadata:

```
{
  "gene_sets": {
    "hsa00010": ["1", "2", "3"],
```



```

        "hsa00020": ["4", "5", "6"]
    },
    "pathway_names": {
        "hsa00010": "Glycolysis / Gluconeogenesis",
        "hsa00020": "Citrate cycle (TCA cycle)"
    },
    "categories": {
        "signal": ["hsa04010", "hsa04012"],
        "metabolic": ["hsa00010", "hsa00020"],
        "disease": ["hsa05200", "hsa05210"]
    }
}

```

[Loading:]

```

import json

# Simple format
with open('gene_sets.json') as f:
    gene_sets = json.load(f)

# With metadata
with open('kegg_pathways.json') as f:
    data = json.load(f)
    gene_sets = data['gene_sets']
    names = data.get('pathway_names', {})

```

6.3 Gene ID Mapping

[CSV/TSV Format:]

```

entrez_id,symbol
7157,TP53
672,BRCA1
1956,EGFR
2064,ERBB2

```

[Requirements:]

- Two columns: entrezid, symbol
- No header required (will be assigned)
- Can have additional columns (ignored)

6.4 Sample Metadata

While not required, we recommend keeping sample metadata:

```
sample_id,group,batch,replicate
sample1,control,batch1,rep1
sample2,control,batch1,rep2
sample3,control,batch2,rep1
sample4,treatment,batch1,rep1
sample5,treatment,batch1,rep2
sample6,treatment,batch2,rep1
```

Use this to define "refindices" and "sampindices":

```
import polars as pl

metadata = pl.read_csv('sample_metadata.csv')
ref_indices = metadata.filter(pl.col('group') == 'control')['sample_id'].to_list()
samp_indices = metadata.filter(pl.col('group') == 'treatment')['sample_id'].to_list()

# Convert to column indices
ref_cols = [expr_data.columns.index(s) for s in ref_indices]
samp_cols = [expr_data.columns.index(s) for s in samp_indices]
```

7. Complete Workflows

7.1 Basic Paired Design

[Scenario:] 3 control vs 3 treatment samples, paired by individual.

```
import polars as pl
import json
from pathlib import Path
from gage_core import GAGEPreparation, GAGEAnalysis

# 1. Load data
expr_data = pl.read_csv('expression_data.csv')
with open('kegg_pathways.json') as f:
    pathways = json.load(f)
gene_sets = pathways['gene_sets']

# 2. Define groups (paired: same individual)
ref_indices = [0, 2, 4] # Control: columns 0, 2, 4
samp_indices = [1, 3, 5] # Treatment: columns 1, 3, 5

# 3. Prepare data
prep = GAGEPreparation()
prepared = prep.prepare_expression(
    expr_data,
    ref_indices=ref_indices,
    samp_indices=samp_indices,
    comparison='paired',
    use_fold=True
)

# 4. Run GAGE
gage = GAGEAnalysis()
results = gage.run_gage(
    prepared,
    gene_sets,
    gene_col='gene_id',
    test_method='t-test'
)

# 5. Filter significant (q < 0.1)
sig = gage.filter_significant(cutoff=0.1)

# 6. Save results
output_dir = Path('results')
output_dir.mkdir(exist_ok=True)

sig['greater'].write_csv(output_dir / 'upregulated_pathways.tsv', separator='\t')
sig['less'].write_csv(output_dir / 'downregulated_pathways.tsv', separator='\t')
```

```
# 7. Print summary
print(f"Up-regulated pathways: {sig['greater'].shape[0]}")
print(f"Down-regulated pathways: {sig['less'].shape[0]}")
print("\nTop 5 up-regulated:")
print(sig['greater'].head(5))
```

7.2 Unpaired Design

[Scenario:] Different individuals in control and treatment groups.

```
# Same setup as above, but change comparison type
prepared = prep.prepare_expression(
    expr_data,
    ref_indices=[0, 1, 2],      # 3 control individuals
    samp_indices=[3, 4, 5],     # 3 different treatment individuals
    comparison='unpaired',      # All 3x3=9 pairwise comparisons
    use_fold=True
)

# Rest is the same
results = gage.run_gage(prepared, gene_sets)
```

7.3 Group Comparison

[Scenario:] Compare average of controls vs average of treatments.

```
prepared = prep.prepare_expression(
    expr_data,
    ref_indices=[0, 1, 2, 3, 4], # 5 controls
    samp_indices=[5, 6, 7, 8, 9], # 5 treatments
    comparison='as.group',        # Mean vs mean
    use_fold=True
)

results = gage.run_gage(prepared, gene_sets)
```

7.4 Time Series Analysis

[Scenario:] Multiple timepoints vs baseline.

```
# Time series: T0 (baseline) vs T1, T2, T3
prepared = prep.prepare_expression(
    expr_data,
    ref_indices=[0, 1, 2],      # T0 (baseline)
```

```

samp_indices=[3, 4, 5,      # T1
              6, 7, 8,      # T2
              9, 10, 11],   # T3
comparison='longgroup',    # Each timepoint vs baseline mean
use_fold=True
)

results = gage.run_gage(prepared, gene_sets)

# Results will have columns for each timepoint
# Can track pathway changes over time

```

7.5 Multi-Study Meta-Analysis

[Scenario:] Combine results from multiple independent studies.

```

from results_analysis import ResultsComparator
from pathlib import Path

# Run GAGE on each study separately
studies = ['study1', 'study2', 'study3']
study_results = []

for study in studies:
    # Load study-specific data
    expr = pl.read_csv(f'{study}_expression.csv')

    # Run GAGE
    prep = GAGEPreparation()
    prepared = prep.prepare_expression(expr, ...)

    gage = GAGEAnalysis()
    results = gage.run_gage(prepared, gene_sets)

    # Save results
    results['greater'].write_csv(f'{study}_greater.tsv', separator='\t')
    study_results.append(Path(f'{study}_greater.tsv'))

# Compare across studies
comparator = ResultsComparator()
combined = comparator.compare_results(
    result_files=study_results,
    sample_names=studies,
    q_cutoff=0.1,
    output_file=Path('meta_analysis.tsv')
)

# Create Venn diagram (if 2-3 studies)
if len(studies) in [2, 3]:
    comparator.create_venn_comparison(
        result_files=study_results,

```

```

        sample_names=studies,
        output_file=Path('meta_venn.png')
    )

# Find consistently significant pathways
consistent = combined.filter(pl.col('hits') == len(studies))
print(f"Pathways significant in all {len(studies)} studies:")
print(consistent)

```

7.6 Complete Analysis Pipeline

[Full workflow from raw data to publication figures.]

```

#!/usr/bin/env python3
"""
Complete GAGE Analysis Pipeline
"""

import polars as pl
import json
from pathlib import Path
from gene_id_utils import GeneIDConverter
from pathway_database_utils import KEGGPathwayRetriever
from gage_core import GAGEPreparation, GAGEAnalysis
from data_processing_utils import GeneExtractor, GeneDataExporter
from visualization_utils import HeatmapPlotter
from results_analysis import SignificanceFilter, GeneSetGroupier

# Configuration
OUTPUT_DIR = Path('gage_analysis_results')
OUTPUT_DIR.mkdir(exist_ok=True)

# 1. Load and prepare data
print("Loading expression data...")
expr_data = pl.read_csv('expression_data.csv')

# 2. Retrieve latest KEGG pathways
print("Retrieving KEGG pathways...")
kegg = KEGGPathwayRetriever()
pathways = kegg.get_pathway_genes(species='hsa', id_type='entrez')
gene_sets = pathways['gene_sets']

# Save for future use
with open(OUTPUT_DIR / 'kegg_pathways.json', 'w') as f:
    json.dump(pathways, f, indent=2)

# 3. Prepare expression data
print("Preparing expression data...")
prep = GAGEPreparation()
prepared = prep.prepare_expression(
    expr_data,

```

```

        ref_indices=[0, 1, 2],
        samp_indices=[3, 4, 5],
        comparison='paired',
        use_fold=True
    )

# 4. Run GAGE analysis
print("Running GAGE analysis...")
gage = GAGEAnalysis()
results = gage.run_gage(
    prepared,
    gene_sets,
    gene_col='gene_id',
    test_method='t-test'
)

# 5. Filter significant pathways
print("Filtering significant pathways...")
sig = gage.filter_significant(cutoff=0.1)

# Save all results
sig['greater'].write_csv(OUTPUT_DIR / 'upregulated_pathways.tsv', separator='\t')
sig['less'].write_csv(OUTPUT_DIR / 'downregulated_pathways.tsv', separator='\t')
sig['stats'].write_csv(OUTPUT_DIR / 'pathway_statistics.tsv', separator='\t')

# 6. Group redundant pathways
print("Grouping redundant pathways...")
grouper = GeneSetGrouper()
groups = grouper.group_gene_sets(
    results=sig['greater'],
    gene_sets=gene_sets,
    expression_data=expr_data,
    output_file=OUTPUT_DIR / 'pathway_groups.json'
)

# 7. Extract essential genes from top pathway
print("Extracting essential genes...")
if sig['greater'].shape[0] > 0:
    top_pathway = sig['greater'][0, 'gene_set']
    pathway_genes = gene_sets[top_pathway]

    extractor = GeneExtractor()
    essential = extractor.extract_essential_genes(
        pathway_genes,
        expr_data,
        threshold=2.0
    )

# Export essential genes with heatmap
exporter = GeneDataExporter()
exporter.export_gene_data(
    genes=essential['gene_id'].to_list(),
    expression_data=expr_data,
    output_file=OUTPUT_DIR / 'essential_genes.csv',

```

```
        create_heatmap=True,
        heatmap_output=OUTPUT_DIR / 'essential_genes_heatmap.png',
        normalize=True
    )

# 8. Create pathway heatmap
print("Creating pathway heatmap...")
if sig['greater'].shape[0] > 1:
    ... [truncated]
```


8. Visualization

8.1 Pathway Heatmaps

[Purpose:] Visualize pathway perturbations across samples.

```
from visualization_utils import HeatmapPlotter
import polars as pl

# Load significant pathways
sig_pathways = pl.read_csv('upregulated_pathways.tsv', separator='\t')

# Select top 30 pathways
top_pathways = sig_pathways.head(30)

# Create heatmap of statistics
plotter = HeatmapPlotter()
plotter.plot_clustered_heatmap(
    data=top_pathways.select([col for col in top_pathways.columns if col.startswith('stat')]),
    row_labels=top_pathways['gene_set'].to_list(),
    cmap='RdYlGn_r',
    figsize=(10, 12),
    output_file='pathways_heatmap.png',
    title='Top 30 Pathways Across Samples'
)
```

8.2 Gene Expression Heatmaps

[Purpose:] Visualize expression of genes in a pathway.

```
from data_processing_utils import DataTransformer, GeneDataExporter
import json

# Load gene sets
with open('kegg_pathways.json') as f:
    pathways = json.load(f)

# Get genes from top pathway
top_pathway_id = sig_pathways[0, 'gene_set']
pathway_genes = pathways['gene_sets'][top_pathway_id]

# Filter expression data to pathway genes
pathway_expr = expr_data.filter(
    pl.col('gene_id').is_in(pathway_genes)
)

# Normalize for visualization
```

```

transformer = DataTransformer()
pathway_expr_norm = transformer.row_normalize(pathway_expr)

# Create heatmap
exporter = GeneDataExporter()
exporter.export_gene_data(
    genes=pathway_genes,
    expression_data=expr_data,
    output_file='pathway_genes.csv',
    create_heatmap=True,
    heatmap_output='pathway_heatmap.png',
    normalize=True
)

```

8.3 Venn Diagrams

[Purpose:] Compare significant pathways across conditions.

```

from results_analysis import ResultsComparator

# Compare two experiments
comparator = ResultsComparator()
comparator.create_venn_comparison(
    result_files=[
        Path('condition1_greater.tsv'),
        Path('condition2_greater.tsv')
    ],
    sample_names=['Condition 1', 'Condition 2'],
    q_cutoff=0.1,
    output_file=Path('conditions_venn.png')
)

```

8.4 Volcano Plots

[Purpose:] Visualize magnitude vs significance of pathway changes.

```

import matplotlib.pyplot as plt
import numpy as np

# Load results
results = pl.read_csv('upregulated_pathways.tsv', separator='\t')

# Extract data
stat_mean = results['stat_mean'].to_numpy()
p_val = results['p_greater'].to_numpy()
q_val = results['q_greater'].to_numpy()

# Create volcano plot

```

```

fig, ax = plt.subplots(figsize=(10, 8))

# Plot all pathways
ax.scatter(stat_mean, -np.log10(p_val), alpha=0.5, s=20, color='gray')

# Highlight significant (q < 0.1)
sig_mask = q_val < 0.1
ax.scatter(
    stat_mean[sig_mask],
    -np.log10(p_val[sig_mask]),
    alpha=0.7,
    s=30,
    color='red',
    label='q < 0.1'
)

# Add threshold lines
ax.axhline(-np.log10(0.05), color='blue', linestyle='--', alpha=0.5, label='p = 0.05')
ax.axhline(-np.log10(0.01), color='blue', linestyle='--', alpha=0.5, label='p = 0.01')

ax.set_xlabel('Mean Test Statistic', fontsize=12)
ax.set_ylabel('-log10(p-value)', fontsize=12)
ax.set_title('Pathway Enrichment Volcano Plot', fontsize=14, weight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('volcano_plot.png', dpi=300)
plt.close()

```

8.5 Pathway Network Visualization

[Purpose:] Visualize relationships between significant pathways.

```

from results_analysis import GeneSetGrouper
import json
import matplotlib.pyplot as plt
import networkx as nx

# Group pathways by overlap
grouper = GeneSetGrouper()
groups = grouper.group_gene_sets(
    results=sig_pathways,
    gene_sets=pathways['gene_sets'],
    expression_data=expr_data,
    p_cutoff=0.05,
    overlap_cutoff=1e-10
)

# Create network graph
G = nx.Graph()

```

```

# Add nodes (pathways)
for group_name, members in groups.items():
    for pathway in members:
        G.add_node(pathway)

# Add edges (significant overlaps)
for group_name, members in groups.items():
    for i, p1 in enumerate(members):
        for p2 in members[i+1:]:
            G.add_edge(p1, p2)

# Plot network
fig, ax = plt.subplots(figsize=(12, 12))
pos = nx.spring_layout(G, k=0.5, iterations=50)

nx.draw_networkx_nodes(G, pos, node_size=300, node_color='lightblue', ax=ax)
nx.draw_networkx_edges(G, pos, alpha=0.3, ax=ax)
nx.draw_networkx_labels(G, pos, font_size=8, ax=ax)

ax.set_title('Pathway Overlap Network', fontsize=14, weight='bold')
ax.axis('off')

plt.tight_layout()
plt.savefig('pathway_network.png', dpi=300, bbox_inches='tight')
plt.close()

```

9. Advanced Topics

9.1 Custom Gene Sets

[Create custom gene sets from biological knowledge:]

```
# Define custom gene sets
custom_sets = {
    'DNA_Repair': ['TP53', 'BRCA1', 'BRCA2', 'ATM', 'ATR', 'CHEK1', 'CHEK2'],
    'Apoptosis': ['BCL2', 'BAX', 'CASP3', 'CASP8', 'CASP9', 'FAS', 'FADD'],
    'Cell_Cycle': ['CDK1', 'CDK2', 'CDK4', 'CCNA1', 'CCNB1', 'CCND1', 'CCNE1']
}

# Convert gene symbols to Entrez IDs if needed
from gene_id_utils import GeneIDConverter

converter = GeneIDConverter('egSymb.csv')
custom_sets_entrez = {}

for set_name, symbols in custom_sets.items():
    entrez_ids = converter.sym2eg(symbols)
    # Filter out None values
    custom_sets_entrez[set_name] = [eid for eid in entrez_ids if eid is not None]

# Use in GAGE analysis
results = gage.run_gage(prepared_data, custom_sets_entrez)
```

9.2 Batch Effect Correction

[Correct for batch effects before GAGE:]

```
from sklearn.preprocessing import StandardScaler
import numpy as np

def correct_batch_effects(expr_data, batch_col):
    """
    Simple batch effect correction using z-score normalization per batch.
    For more sophisticated methods, use Combat or SVA.
    """
    # Separate by batch
    batches = expr_data[batch_col].unique().to_list()

    corrected_dfs = []
    for batch in batches:
        batch_data = expr_data.filter(pl.col(batch_col) == batch)

        # Get numeric columns
```

```

numeric_cols = batch_data.select(pl.col(pl.NUMERIC_DTYPES)).columns

# Z-score normalize
for col in numeric_cols:
    values = batch_data[col].to_numpy()
    scaler = StandardScaler()
    normalized = scaler.fit_transform(values.reshape(-1, 1)).flatten()
    batch_data = batch_data.with_columns(
        pl.Series(col, normalized)
    )

corrected_dfs.append(batch_data)

return pl.concat(corrected_dfs)

# Apply batch correction
corrected_data = correct_batch_effects(expr_data, 'batch')

```

9.3 Filtering Low-Quality Gene Sets

[Remove gene sets with poor quality or insufficient coverage:]

```

def filter_gene_sets(gene_sets, expr_data, gene_col='gene_id',
                    min_coverage=0.5, min_size=10, max_size=500):
    """
    Filter gene sets based on coverage and size.

    Parameters:
    - min_coverage: Minimum fraction of genes present in expression data
    - min_size: Minimum gene set size
    - max_size: Maximum gene set size
    """
    # Get available genes
    available_genes = set(expr_data[gene_col].to_list())

    filtered_sets = {}
    stats = []

    for set_name, genes in gene_sets.items():
        # Check size
        if len(genes) < min_size or len(genes) > max_size:
            continue

        # Check coverage
        present_genes = [g for g in genes if g in available_genes]
        coverage = len(present_genes) / len(genes)

        if coverage >= min_coverage:
            filtered_sets[set_name] = present_genes
            stats.append({
                'set_name': set_name,

```

```

        'original_size': len(genes),
        'filtered_size': len(present_genes),
        'coverage': coverage
    })

    # Print summary
    stats_df = pl.DataFrame(stats)
    print(f"Filtered {len(gene_sets)} → {len(filtered_sets)} gene sets")
    print(f"Mean coverage: {stats_df['coverage'].mean():.2%}")

    return filtered_sets

# Apply filtering
filtered_gene_sets = filter_gene_sets(
    gene_sets,
    expr_data,
    min_coverage=0.7, # 70% of genes must be present
    min_size=15,
    max_size=300
)

```

9.4 Permutation Testing

[Assess significance using permutation tests:]

```

import numpy as np
from gage_core import GAGEAnalysis

def permutation_test(expr_data, gene_sets, ref_indices, samp_indices,
                    n_permutations=1000, seed=42):
    """
    Permutation test for GAGE results.

    Returns empirical p-values based on permuted data.
    """
    np.random.seed(seed)

    # Run original analysis
    prep = GAGEPreparation()
    prepared = prep.prepare_expression(
        expr_data,
        ref_indices=ref_indices,
        samp_indices=samp_indices,
        comparison='paired'
    )

    gage = GAGEAnalysis()
    original_results = gage.run_gage(prepared, gene_sets)
    original_stats = original_results['greater']['stat_mean'].to_numpy()

    # Permutation testing

```

```

permuted_stats = np.zeros((n_permutations, len(original_stats)))

all_indices = ref_indices + samp_indices

for i in range(n_permutations):
    # Permute sample labels
    perm_indices = np.random.permutation(all_indices)
    n_ref = len(ref_indices)

    perm_ref = perm_indices[:n_ref].tolist()
    perm_samp = perm_indices[n_ref:].tolist()

    # Run GAGE on permuted data
    perm_prepared = prep.prepare_expression(
        expr_data,
        ref_indices=perm_ref,
        samp_indices=perm_samp,
        comparison='paired'
    )

    perm_results = gage.run_gage(perm_prepared, gene_sets)
    permuted_stats[i, :] = perm_results['greater']['stat_mean'].to_numpy()

# Calculate empirical p-values
empirical_p = np.mean(permuted_stats >= original_stats[:, None], axis=1)

# Add to results
results_with_perm = original_results['greater'].with_columns(
    pl.Series('empirical_p', empirical_p)
)

return results_with_perm

# Run permutation test
perm_results = permutation_test(
    expr_data,
    gene_sets,
    ref_indices=[0, 1, 2],
    samp_indices=[3, 4, 5],
    n_permutations=1000
)

print(perm_results.head(10))

```

9.5 Integration with Single-Cell Data

[Aggregate single-cell data for GAGE analysis:]

```

def aggregate_sc_data(sc_expr, cell_types, gene_col='gene_id'):
    """
    Aggregate single-cell expression data by cell type.

```



```

Parameters:
- sc_expr: Single-cell expression DataFrame (genes × cells)
- cell_types: Cell type annotations for each cell
- gene_col: Gene ID column name

Returns:
- Aggregated expression DataFrame (genes × cell_types)
"""
# Get unique cell types
unique_types = sorted(set(cell_types))

# Get gene column
genes = sc_expr[gene_col]

# Aggregate by cell type (mean expression)
aggregated_data = {gene_col: genes}

for cell_type in unique_types:
    # Get cells of this type
    type_mask = [ct == cell_type for ct in cell_types]
    type_indices = [i for i, m in enumerate(type_mask) if m]

    # Get columns for this cell type (excluding gene_col)
    cell_cols = [sc_expr.columns[i+1] for i in type_indices]

    # Calculate mean expression
    type_expr = sc_expr.select(cell_cols).mean_horizontal()
    aggregated_data[cell_type] = type_expr

return pl.DataFrame(aggregated_data)

# Example usage
# sc_data: genes × 10000 cells
# cell_annotations: list of 10000 cell type labels

aggregated = aggregate_sc_data(
    sc_expr=sc_data,
    cell_types=cell_annotations
)

# Now use aggregated data for GAGE
# Compare different cell types
results = gage.run_gage(
    prepared_agg_data,
    gene_sets,
    # e.g., T cells vs B cells
)

```

10. Troubleshooting

10.1 Common Errors

Error: "No numeric columns found"

[Cause:] Expression data doesn't have numeric columns or gene ID column is not excluded.

[Solution:]

```
# Check columns
print(expr_data.columns)
print(expr_data.dtypes)

# Ensure gene_col is specified correctly
results = gage.run_gage(
    prepared,
    gene_sets,
    gene_col='gene_id' # Must match your column name
)
```

Error: "Mapping data not loaded"

[Cause:] GeneIDConverter initialized without mapping file.

[Solution:]

```
# Initialize with mapping file
converter = GeneIDConverter('egSymb.csv')

# Or load after initialization
converter.load_mapping('egSymb.csv')
```

Error: "Can't create Venn diagram for more than 3 sets"

[Cause:] Venn diagrams only support 2-3 comparisons.

[Solution:]

```
# For >3 comparisons, use upset plot or heatmap instead
# Or compare pairwise
```

Warning: "No genes found in gene set"

[Cause:] Gene IDs don't match between expression data and gene sets.

[Solution:]

```
# Check ID systems match
print("Expression genes:", expr_data['gene_id'].head())
print("Gene set genes:", list(gene_sets.values())[0][:5])

# Convert if needed
from gene_id_utils import GeneIDConverter
converter = GeneIDConverter('egSymb.csv')

# Convert gene sets to match expression data
converted_sets = {}
for name, genes in gene_sets.items():
    converted_sets[name] = converter.eg2sym(genes)
```

10.2 Performance Issues

Slow analysis with many gene sets

[Solution:]

```
# Filter gene sets before analysis
filtered_sets = {
    k: v for k, v in gene_sets.items()
    if 10 <= len(v) <= 500 # Only keep reasonable sizes
}

# Or use parallel processing (future feature)
```

Memory errors with large datasets

[Solution:]

```
# Use polars lazy evaluation
expr_lazy = pl.scan_csv('large_data.csv')

# Process in chunks
chunk_size = 1000
for i in range(0, len(gene_sets), chunk_size):
    chunk_sets = dict(list(gene_sets.items())[i:i+chunk_size])
    chunk_results = gage.run_gage(prepared, chunk_sets)
    # Save chunk results
```

10.3 Statistical Issues

Too many/too few significant gene sets

[Causes and Solutions:]

[Too many significant:]

Lower cutoff (e.g., 0.05 instead of 0.1)

Use stricter test method

Check for batch effects or confounders

[Too few significant:]

Increase cutoff (e.g., 0.25 instead of 0.1)

Check if sample size is sufficient

Verify experimental design is correct

Try different comparison methods

Dual-significant gene sets

[Cause:] Gene sets significant in both directions.

[Solution:]

```
# Use Stouffer's method (default in PyGAGE)
# Or filter with dual_sig parameter
from results_analysis import SignificanceFilter

filterer = SignificanceFilter()
filtered = filterer.filter_significant(
    results,
    dual_sig=1 # Keep only more significant direction
)
```

10.4 Data Quality Issues

Check expression data quality

```
import matplotlib.pyplot as plt
import numpy as np

# Check for outliers
```

```

sample_means = expr_data.select(pl.col(pl.NUMERIC_DTYPES)).mean().transpose()
print("Sample means:", sample_means)

# Plot distribution
fig, ax = plt.subplots(figsize=(10, 6))
for col in expr_data.columns[1:]: # Skip gene_id
    values = expr_data[col].to_numpy()
    ax.hist(values, bins=50, alpha=0.3, label=col)

ax.set_xlabel('Expression Value')
ax.set_ylabel('Frequency')
ax.set_title('Expression Value Distributions')
ax.legend()
plt.show()

# Check for missing values
missing = expr_data.null_count()
print("Missing values per column:", missing)

```

Filter low-expressed genes

```

def filter_low_expression(expr_data, min_expr=1.0, min_samples=3):
    """
    Remove genes with low expression across all samples.

    Parameters:
    - min_expr: Minimum expression threshold
    - min_samples: Minimum number of samples above threshold
    """
    numeric_cols = expr_data.select(pl.col(pl.NUMERIC_DTYPES)).columns

    # Count samples above threshold for each gene
    above_threshold = (
        expr_data.select(numeric_cols) > min_expr
    ).sum_horizontal()

    # Filter genes
    filtered = expr_data.filter(above_threshold >= min_samples)

    print(f"Filtered {expr_data.shape[0]} → {filtered.shape[0]} genes")
    return filtered

# Apply filtering
filtered_expr = filter_low_expression(expr_data, min_expr=1.0, min_samples=3)

```

11. References

Primary Citation

Luo, W., Friedman, M., Shedden K., Hankenson, K. and Woolf, P. (2009)
[GAGE: Generally Applicable Gene Set Enrichment for Pathway Analysis.]
BMC Bioinformatics 10:161
<https://doi.org/10.1186/1471-2105-10-161>

Related Methods

[Gene Set Enrichment Analysis (GSEA):]
Subramanian, A. et al. (2005)
Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles.
PNAS 102(43):15545-15550

[Parametric Analysis of Gene Set Enrichment (PAGE):]
Kim, S. Y. and Volsky, D. J. (2005)
PAGE: Parametric analysis of gene set enrichment.
BMC Bioinformatics 6:144

Statistical Methods

[Stouffer's Method:]
Stouffer, S. A. et al. (1949)
The American Soldier: Adjustment during Army Life.
Princeton University Press

[Benjamini-Hochberg Procedure:]
Benjamini, Y. and Hochberg, Y. (1995)
Controlling the false discovery rate: A practical and powerful approach to multiple testing.
Journal of the Royal Statistical Society: Series B 57(1):289-300

Databases

[KEGG:]

Kanehisa, M. and Goto, S. (2000)

KEGG: Kyoto Encyclopedia of Genes and Genomes.

Nucleic Acids Research 28:27-30

<https://www.genome.jp/kegg/>

[Gene Ontology:]

The Gene Ontology Consortium (2000)

Gene Ontology: Tool for the unification of biology.

Nature Genetics 25:25-29

<http://geneontology.org/>

Python Libraries

[Polars:]

<https://pola.rs/>

[SciPy:]

<https://scipy.org/>

[Matplotlib:]

<https://matplotlib.org/>

[Seaborn:]

<https://seaborn.pydata.org/>

Appendix A: Command-Line Reference

Complete CLI Options

gagecore.py

```
usage: gage_core.py [-h] --expression EXPRESSION --gene-sets GENE_SETS
                  [--gene-col GENE_COL]
                  [--ref-indices REF_INDICES [REF_INDICES ...]]
                  [--samp-indices SAMP_INDICES [SAMP_INDICES ...]]
                  [--comparison {paired,unpaired,1ongroup,as.group}]
                  [--test-method {t-test,z-test,ks-test}]
                  [--cutoff CUTOFF] --output OUTPUT
```

Run GAGE analysis

required arguments:

--expression EXPRESSION	Expression data file (CSV/TSV)
--gene-sets GENE_SETS	Gene sets JSON file
--output OUTPUT	Output directory for results

optional arguments:

--gene-col GENE_COL	Gene ID column name (default: gene_id)
--ref-indices REF_INDICES [REF_INDICES ...]	Reference sample column indices
--samp-indices SAMP_INDICES [SAMP_INDICES ...]	Sample column indices
--comparison {paired,unpaired,1ongroup,as.group}	Comparison type (default: paired)
--test-method {t-test,z-test,ks-test}	Statistical test method (default: t-test)
--cutoff CUTOFF	Q-value cutoff for significance (default: 0.1)

resultsanalysis.py compare

```
usage: results_analysis.py compare [-h] --inputs INPUTS [INPUTS ...]
                                   --names NAMES [NAMES ...]
                                   [--cutoff CUTOFF] --output OUTPUT
                                   [--venn VENN]
```

Compare GAGE results across multiple datasets

required arguments:

```
--inputs INPUTS [INPUTS ...]      Input result files
--names NAMES [NAMES ...]         Sample names
--output OUTPUT                   Output file
```

optional arguments:

```
--cutoff CUTOFF                  Q-value cutoff (default: 0.1)
--venn VENN                      Create Venn diagram (output image file)
```

Appendix B: Example Datasets

Example datasets are available for testing PyGAGE:

Small Test Dataset

[expressiontest.csv] (100 genes × 6 samples)

- 3 control samples (columns 0-2)
- 3 treatment samples (columns 3-5)
- Simulated differential expression

[genesetstest.json] (20 gene sets)

- 10-50 genes per set
- Includes both enriched and non-enriched sets

Real Dataset Example

[GSE16873] - Breast cancer study

- Normal breast tissue vs DCIS (ductal carcinoma in situ)
- Available from GEO: <https://www.ncbi.nlm.nih.gov/geo/>

Appendix C: Glossary

[Gene Set:] A collection of genes grouped by biological function, pathway, or experimental annotation.

[Enrichment:] Statistical over-representation of a gene set in differential expression results.

[FDR (False Discovery Rate):] Expected proportion of false positives among rejected hypotheses.

[Q-value:] FDR-adjusted p-value.

[Fold Change:] Ratio of expression between conditions (typically log2-transformed).

[Same Direction Test:] Tests whether genes in a set are consistently up- or down-regulated.

[Two-Direction Test:] Tests whether genes in a set are perturbed regardless of direction.

[Dual-Significant:] Gene set significant in both up and down directions (usually unwanted).

[Essential Genes:] Genes in a set that contribute most to the enrichment signal.

[Gene Set Size:] Number of genes in a gene set; typically filtered to 10-500 genes.

[End of PyGAGE User Manual]

For questions, bug reports, or feature requests, please contact:

[Your contact information]

Last updated: February 2026